

Deseño físico

Almacenamento de datos

Zonas lóxicas e físicas

Unha zona lóxica (ou espazo lóxico) de almacenamento de datos inclúe unha ou varias zonas físicas de almacenamento de datos.

Cando se crea unha táboa, pode especificarse a zona lóxica, non a física.

En xeral, unha zona física é un ficheiro do sistema operativo, pero hai alternativas: dispositivos ou particións (*raw devices*), xestores de volumes como Oracle ASM (crea sistemas de ficheiros propios), ...

A nomenclatura varía co SXBD: unha zona lóxica pode chamabrse *tablespace* (Oracle, Mysql/MariaBD, PostgreSQL) ou *filegroup* (MS SQL Server), por exemplo

Oracle

```
-- Oracle
CREATE TABLESPACE TS1
  ADD DATAFILE '/u01/data/ts1.dbf'
  SIZE 200M AUTOEXTEND ON;

CREATE TABLE EMP(
  EMPNO NUMBER(3) ....
) TABLESPACE TS1;
```

MS SQL SERVER

```
-- MS SQL Server
ALTER DATABASE bd1 ADD FILEGROUP FG1;
ALTER DATABASE bd1 ADD FILE
( NAME = fg01,
  FILENAME = 'C:\Data\fg01.ndf',
  SIZE = 200MB, FILEGROWTH = 5MB)
TO FILEGROUP FG1;

CREATE TABLE EMP(
  EMPNO NUMBER(3) ....
) ON FG1;
```

Espazo asociado a un obxecto

Foto de la diapositiva 7

Algúns xestores (ex: Oracle) usan o nome **segmento** para referirse ós datos almacenados por un obxecto (táboa, índice) (salvo particionado de datos).

Un segmento, ou zona de almacenamento dun obxecto, pode estar almacenado en varios ficheiros, pero nunha sóa zona lóxica de almacenamento.

Un segmento está formado por **extensións** (extents), que son un conxunto contiguo de bloques (nun único ficheiro).

As extensións son a unidade de asignación de espazo, poden ser de tamaño fixo ou variable (depende do SXBD).

Un bloque almacena filas, que habitualmente teñen tamaño variable (rexistros de lonxitude variable, *slotted pages*).

O tamaño do bloque é un múltiplo do tamaño do bloque do sistema de ficheiros. Dependendo do xestor pode ser fixo ou variable. Un tamaño habitual é 8KB.

Exemplo Oracle

En Oracle, un tablespace está formado por un conxunto de *data files* (*temp files* se é temporal).

Tablespaces que Oracle crea de forma automática

- **SYSTEM** : Catálogo do sistema, táboas e vistas para funcións administrativas, obxectos compilados, ...
- **SYSAUX** : Auxiliar do anterior, usado por diversos componentes de Oracle.
- **UNDOTBS1** : Para rexistros de desfacer (undo), necesario para rollbacks e para consistencia de lectura.
- **TEMP** : Tablespace temporal.
- **USERS** : Para obxectos dos usuarios.

Organización das táboas

A organización máis común é a **heap table** (táboa apilada).

- As filas engádense en calquera bloque onde haxa espazo.
- Non hai ningún tipo de ordenación das filas.
- Normalmente as insercións son moi eficientes.
- Poden quedar ocos debido a sucesivas operacións (insercións, borrados e actualizacións).
- A búsqueda non é eficiente (pode compensarse coa creación de índices).

Outras posibles organizacións (verémolas en detalle máis adiante) inclúen:

- Táboas almacenadas como índices (ordenadas pola clave primaria), como *IOT* (Index Organized Table) en Oracle ou *clustered index* en MS SQL Server.
- Clusters: permiten almacenar varias táboas nos mesmos bloques (ex: facturas coas súas liñas), podendo utilizar a baixo nivel táboas apiladas máis índices ou unha táboa hash.

Índices

Acceso a datos (sen índices)

Selectividade dunha consulta

A selectividade dunha consulta (ou predicado) é inversamente proporcional á porcentaxe de filas que devolve. Unha selectividade alta devolverá unha porcentaxe baixa de filas, e viceversa.

Exemplo: `T(id, val)` con 100.000 filas. Campo id: vai de 1 a 100.000. Campo val: valores 1 e 2 distribuídos uniformemente.

- Un predicado `val=1` terá unha baixa selectividade, xa que devolve aproximadamente o 50% das filas.
- Na mesma táboa, un predicado `id = 20` terá unha altísima selectividade (só unha fila, un 0.001% do total), pero `id > 20` terá unha selectividade moi baixa, xa que devolve máis do 99.9% das filas.

Baixa selectividade:

- Debe lerse toda a zona de almacenamiento de forma secuencial.
- É eficiente.

Alta selectividade:

- Debe lerse toda a zona de almacenamento (salvo condicións de unicidade). Realízanse moitas lecturas que non producen resultado.
- É moi pouco eficiente

Conceptos de índice

- Un índice en bases de datos é análogo a un índice alfabético dun libro de texto, onde para cada concepto se indican aquelas páxinas nas que aparece. A alternativa a buscar no índice e ir ás páxinas indicadas sería ler o libro completo para atopar o concepto buscado.
- Partimos da existencia dunha táboa (habitualmente unha *heap table*, pero non é obrigatorio).

- O índice é unha estrutura auxiliar que permite axilizar certas operacións sobre unha táboa. Ofrece un camiño alternativo para acceder ós datos sen ter que realizar unha exploración completa da táboa (evita accesos innecesarios a zonas de almacenamiento da táboa).
- Un índice almacena entradas de índice, compostas por valores da **clave de indexación** (formada por un ou máis campos da táboa) xunto coa posición física que ocupa. Habitualmente será o enderezo físico da fila (`rowid` ou `rid`). O índice estará habitualmente ordenado pola clave de indexación.
- O acceso a datos a través dun índice implica buscar as entradas no índice e logo acceder á táboa a través dos `rowids`. Ambas operacións son moi rápidas

Índices en árbore - Árbores B+

- Existen distintos tipos de índices lineais (ex: hash), pero os máis habituais son en forma de árbore.
- Destaca fundamentalmente a árbore B+.
 - É unha árbore balanceada (as follas están todas ó mesmo nivel).
 - Os nodos folla conteñen as claves de indexación xunto coas `rowid` para localizar as filas). Todo valor da clave de indexación aparecerá nun nodo folla. Os nodos folla están enlazados (listas enlazada simple, ou dobre nalgunha implementación).
 - Os valores das claves nos nodos intermedios están tamén nos nodos folla. Utilízanse para navegar pola árbore

Foto de la diapositiva 13

Acceso a datos a través de índices

Consultas con alta selectividade:

- Accédese ao índice e logo a unha zona reducida da táboa (lecturas diferenciadas).
- Eficiente.

Consultas con baixa selectividade:

- Accedese ao índice (posiblemente varias follas) logo a unha zona ampla da táboa (lecturas diferenciadas).
- Baixa eficiencia.

Creación e borrado de índices en SQL

- O estándar SQL non contempla en absoluto os índices.
- Os distintos SXBD poden ter variantes, pero a sintaxe básica de creación e borrado é similar:

```
CREATE [<tipo>] INDEX <nome-índice> ON <táboa> (<campos>);  
DROP INDEX <nome-índice>
```

- É importante ter en conta que os SXBD poden crear índices automaticamente (xeralmente, a partir das restricións).

Por exemplo:

- Cando creamos unha restrición de unicidade ou de clave primaria, todos os SXBD crean un índice único sobre os campos correspondentes.
- Algúns xestores poden crear índices sobre claves foráneas, pero non é tan habitual. Debemos conocer o que fai o SXBD que esteamos utilizando.

Ventaxas e inconvenientes dos índices

Os índices poden mellorar a eficiencia de certas consultas.

- Habitualmente falamos de acelerar os `SELECT`, pero tamén son útiles para `DELETE` e `UPDATE` (hai que localizar as filas a borrar ou modificar).
- A eficiencia baséase en evitar acceder a amplas zonas de disco (táboas) que non conteñen os datos buscados.
- Poden acelerar a recuperación de datos ordenados pola clave de indexación.
- Os índices "habituais" (árbores B+) son normalmente útiles cando a consulta recupera unha porcentaxe baixa de filas, pero hai outros índices que son útiles aínda que se recupere unha porcentaxe alta.

Os índices tamén teñen inconvenientes:

- Ocupan espazo en disco.
- Retardan as operacións DML de modificación de datos.
- Un índice non sempre será axeitado. O optimizador decide se usalo ou non.
- Se o optimizador non utiliza un índice, é contraproducente, xa que o retarde das operacións DML sempre ocorre. Deben eliminarse os índices que non use o optimizador.

Tipos de índices

- Valores únicos ou repetidos na clave de indexación:
 - **Único**: A clave contén valores únicos.
 - **Non único**: A clave admite duplicados (valores repetidos).

```
create UNIQUE index i_dept_dname on dept(dname); -- Único  
create index i_emp_sal on emp(sal); -- Non único
```

- Número de columnas da clave de indexación:

- **Simple:** O índice está definido sobre unha soa columna da táboa.
- **Composto:** Está definido sobre varias columnas da táboa.

A elección da orde dos campos clave de indexación é moi relevante para determinar que tipo de consultas pode acelerar.

```
create index i_emp_sal on emp(sal);           -- Simple
create index i_emp_job_sal on emp(job, sal); -- Composto
```

- Existe unha entrada no índice por cada fila da táboa?
 - **Denso:** Si, existe unha entrada por cada fila.
 - **Escaso/Disperso:** Non hai unha entrada por cada fila. Implica que debe ser agrupado.
- Os datos da táboa están ordenados?
 - **Non agrupado** ou **secundario:** Os datos da táboa non están ordenados pola clave de indexación. Sobre unha táboa pode haber máis dun índice secundario.
 - **Agrupado:** A táboa está físicamente ordeada pola clave de indexación.
 - Só pode haber un índice agrupado por táboa.
 - É frecuente que se cree sobre a clave primaria da táboa. Neste caso o índice denomínase **primario**.
 - Se é un índice único (sen duplicados, por ser primario ou por outra restrición de unicidade), será denso. Se admite duplicados poder ser un índice disperso (unha entrada de índice por cada valor distinto da clave de indexación).
 - Son normalmente axeitados para optimizar búsquedas por rango, aínda que recuperen unha alta porcentaxe das filas.
 - Caso particular: Os nodos folia do índice almacenan as filas completas da táboa en lugar do `rowid`. Os datos da táboa están almacenados no propio índice.
Oracle: *IOT (Index Organized Table)*; MS SQL Server: *Clustered Index*.

Outras organizacións e índices

Particionado

Particionado de táboas e índices

Descrición xeral

Particionar: consiste en fragmentar unha táboa (ou índice) grande en diferentes fragmentos ou particións máis pequenos e manexables.

- Oracle recomenda particionar aquelas táboas que superen 1GB de tamaño.
- É unha fragmentación *horizontal*, de xeito que filas completas (ou entradas de índice) van a unha ou outra partición.
- O criterio de particionado utilizará unha **clave de particionado** (condicións sobre unha ou varias columnas).
- Cada **partición** será un **segmento**.
- Permítese especificar un tablespace diferente para cada partición.
- Unha vez creada a táboa particionada, poden engadirse ou eliminarse particións.

Tipos de particionado de táboas

- Baseado en rangos: os datos sepáranse en particións baseándose en rangos de valores. É moi utilizado con dominios de datos continuos, e con información temporal.
- Baseado en listas: Utilizado con dominios discretos, especifícanse listas de valores para definir as particións.
Existen limitacións: por exemplo, Oracle só permite particionar mediante listas se a clave de particionado é un único atributo.
- Hash: Indícase un número de particións, e a asignación de filas virá determinada pola aplicación dunha función hash á clave de particionado.
Un atributo con alta cardinalidade (como unha clave primaria) é un bo candidato para o particionado hash.
- Composta (híbrida): Oracle permite o particionado híbrido, onde en primeiro lugar se particiona por rango, e en cada partición créanse subparticións por lista ou hash.

Tipos de particionado de índices

O particionado dun índice pode estar asociado ou non a unha táboa particionada.

- Particionado Local: O índice sigue o mesmo esquema de particionado que a táboa. En cada partición do índice só hai entradas referenciando filas na partición correspondente da táboa.
- Particionado Global: O índice sigue o seu esquema propio de particionado. As entradas dunha partición do índice poden apuntar a filas en varias particións da táboa.
- Sen particionar: É un caso específico de particionado global, cunha soa partición. É a opción predeterminada cando se crea un índice sobre unha táboa particionada.

Ventaxas

- Optimización de consultas descartando particións
 - Interesante para consultas con selectividade media.
Con selectividade alta interesaría un índice; con selectividade baixa un acceso

completo á táboa.

- `SELECT`s que inclúen no `WHERE` o criterio de particionado.
- Teoricamente, podería optimizar joins se ambas táboas están particionadas seguindo o mesmo criterio.
- Maior grado de paralelismo: Cada partición pode almacenarse e está nun tablespace separado, pode poñerse off-line e restaurar datos. As consultas que non necesitan esa partición siguen funcionando con normalidade.
- Melloras para operacións DML masivas:
 - En borrados masivos: menos bloqueos, a veces podemos eliminar a partición completamente.
 - En insercións masivas: por exemplo en entornos de Data Warehouses, cargamos un novo mes de datos nunha partición nova sen bloquear nada do resto.
 - As 2 anteriores implican melloras no *roll-in / roll-out*: engadir/borrar particións para manter na DB unha ventá de tempo, por exemplo os últimos 10 anos dun DW).
- Clusterización implícita polo criterio de particionado. NON ordena, pero agrupa. Só é útil se os criterios de particionado coinciden co criterio de agrupación/ordenación das consultas.
- Con índices totalmente particionados (seguindo o criterio da táboa) temos varios índices, pero máis pequenos, e non hai que reconstruír o índice completo se por exemplo borramos unha partición.
- Melloras en tarefas de administración: Por exemplo, backups se temos a táboa particionada por data.

Inconvenientes

Particionar unha táboa ou índice cun criterio arbitrario non mellora automaticamente a eficiencia. De feito, pode empeorar.

Exemplo:

- Particionamos a táboa `emp_part_hash` por hash na clave primaria (`empno`).
- Creamos un índice sobre o campo `sal` con particionado local. `create index i_sal on emp_part_hash(sal) LOCAL;`
- As consultas como a seguinte deben explorar todos os índices locais en vez dun só índice global (normalmente implica máis operacións de E/S).

```
select *  
  from emp_part_hash  
 where sal > 1000;
```

Clusters (Agrupamentos)

Descrición xeral

Un cluster permite almacenar datos de varias táboas no mesmo bloque físico da base de datos. Está baseado no uso dunha clave de agrupamento (un atributo ou conxunto de atributos común a todas as táboas).

Todos os datos cun determinado valor da clave estarán xuntos, optimamente no mesmo bloque da base de datos.

Note

Non deben confundirse con índice de agrupamento

Tipos de clusters

- **B+ tree cluster:** Utiliza un índice (árbore B+) que almacena os valores da clave e o bloque onde se almacenan as filas das táboas que teñan ese valor. É como un índice normal, pero serve á vez para todas as táboas do cluster.
- **Hash cluster:** Sustitúe o índice por unha función hash aplicada á clave de agrupamento. Oracle preasigna espacio baseado no número (estimado) de valores diferentes para a clave de agrupamento que se lle indique na creación do cluster.

Ventaxas

- Incrementa a eficiencia dos joins (podemos ver o cluster como un join precalculado=).
- Podemos mellorar a eficiencia do buffer chache (menos bloques a manexar xa que nun bloque hai datos de varias táboas).
- Pode reducir a cantidade de índices necesarios: só ún índice (ou ningún no caso de hash cluster) en lugar de ter un índice por táboa.

Inconvenientes

- O escaneo secuencial de só unha das táboas do cluster pode ser máis lento (por ter que examinar máis bloques) que nunha táboa *heap*.
- As insercións son normalmente máis lentas, xa que hai que buscar a ubicación correcta de cada fila.

Outras consideracións

- O grado de clusterización é moi dependente de como se xestioann os datos (para un alto grado de clusterización deben insertarse á vez datos de todas as táboas que compartan unha determinada clave).

- O dimensionamento (`SIZE` e `HASHKEYS`) determinagrandemente a eficiencia. Valores demasiado grandes desperdician espazo, demasiado pequenos diminúen o grado de clusterización. Para correxir malas estimacións habría que recrear o cluster desde cero.
- É posible crear un hash cluster para unha táboa soa (coa opción de Oracle `single table`). É especialmente indicado para *lookup tables*, onde buscamos táboas de códigos para atopas o valor asociado a un código, xa que elimina a necesidade de crear un índice (e os accesos a ese índice en cada búsqueda).

Index-Organized Tables

Descrición xeral

- Unha táboa organizada como índice (IOT, Index.Organized Table) de Oracle é basicamente unha táboa almacenada nun índice.
- Corresponde ao concepto de índice de agrupamento (clustered index) en SQL Server.
- É similar a unha árbore B+ estándar, pero nos nodos folla en lugar de `rowids` almacena, xunto coa clave de indexación, o resto dos datos da fila. En algún caso, por exemplo se a táboa ten moitas columnas ou moi grandes, pode almacenarse parte da fila nun espazo de desbordamento cunha táboa heap.
- A clave de indexación é a clave primaria da táboa.
- As entradas dunha IOT non teñen rowid, porque se moverán ó reestaurar o índice debido a operacións DML.
- Pode crearse un índice secundario sobre unha IOT. Este índice terá como *rowid virtual* o valor da clave primaria da fila na IOT.

```
CREATE TABLE pais(
  id INT CONSTRAINT pk_pais PRIMARY KEY,
  nome VARCHAR(200) NOT NULL,
  poboacion NUMERIC(10)
) ORGANIZATION INDEX;
```

Ventaxas

- Alta eficiencia de búsqueda por clave primaria. Só require acceso ó índice (non hai táboa, polo tanto non hai un acceso a disco adicional vía rowid).
- Aforro de espazo: Non necesitamos a táboa máis un espazo adicional para o índice, todo se garda no índice.
- A ordenación por clave mantense sempre (é un índice). En cambio, nos clusters depende da orde de inserción das filas (o grado de clusterización baixa se as claves iguais non se insertan xuntas).

Inconvenientes

- As operacións DML poden ser máis lentas que nunha táboa *heap*, por exemplo se requiren reestruturación (rebalanceo) da árbore.
- Os índices secundarios son menos eficientes:
 - Sobre unha táboa *heap*, unha búsqueda por índice secundario dun valor require un recorrido da árbore B+ do índice máis un só acceso por rowid.
 - Sobre unha IOT, require un recorrido da árbore B+ do índice máis outro recorrido da árbore B+ que almacena a IOT.